Demystifying Containers

This two-part series is all about understanding what happens behind the scenes when we run a *docker run <image>* command. Let us uncover the magic of container technology by building a container from scratch.

Ever since Docker released its first version back in 2013, it triggered a major shift in the way the software industry works. Lightweight VMs suddenly caught the attention of the world and opened opportunities for unlimited possibilities. Containers provided a way to get a grip on software that was built. Docker containers could be used to wrap up an application in such a way that deployment and runtime issues like how to expose them on a network; how to manage their use of storage, memory and I/O; how to control access permissions, etc, were handled outside of the application itself, and in a way that was consistent across all containerised apps.

Containers offer many other benefits besides just handy encapsulation, isolation, portability and control. They are small in size (megabytes) and can start instantly. They have their own built-in mechanisms for versioning and component reuse. They can be easily shared via public or private repositories.

Today, containers are an essential component of the software development process. Many of us use them on a day-to-day basis. Despite this, there is still a lot of magic involved for many who want to venture into the world of containers in general. To date, there is a lot of ambiguity about how exactly a container works. The two articles in this series will try to demystify the working of containers. But before that, I believe we must understand how containers came to be.

The world before containers

For many years now, enterprise software has typically been deployed either on bare metal (i.e., installed on an operating system that has complete control over the underlying hardware) or in a virtual machine (i.e., installed on an operating system that shares the underlying hardware with other guest operating systems). Naturally, installing on bare metal made the software painfully difficult to move around and difficult to update — two constraints that made it hard for IT teams to respond nimbly to changes in business needs.

Then virtualisation came along. Virtualisation platforms (also known as hypervisors) allowed multiple virtual machines to share a single